



SENIOR SOFTWARE ENGINEER INTERVIEW QUESTIONS: DEEP DIVES

Kafka Observability

The senior engineer answer sheet

Every service green. Every broker healthy. Payments twenty minutes late.

There are three layers worth watching, and most teams build the one that is easiest to build and then wonder why it never catches anything.

Infrastructure says whether Kafka is alive. The pipeline says whether work is flowing. The business layer says whether payments are completing, and completing CORRECTLY. A green first layer says nothing about the third.

Kafka behaviour below is sourced to Apache Kafka 4.3 documentation. Statistical statements are named as statistics. Operational judgements are named as judgements.

In scope

- The three layers, and which one catches what
- Lag, and the two different stories one number tells
- Count against age
- The correctness failure no infrastructure metric sees
- Retry and dead letter signals
- Rebalances on the same timeline as lag
- Percentiles, and the aggregation error almost everyone makes
- Cardinality, and the label that kills a metrics backend

Out of scope

- Specific dashboard IDs, which get deleted and renumbered
- Exporter versions and copy-paste metric paths
- Log aggregation and tracing backends by name
- Broker side capacity metrics and cluster sizing

A metric path in a PDF is wrong within a year. What a signal means is still true in five.

01 Worked assumptions

Payments per second	100
Backlog A	10,000 records, clearing in 2 seconds
Backlog B	10 records, sitting for 2 hours
Which one is the incident	B, and only age shows it
Payments finishing in 100 ms	99 out of 100
The hundredth	10 seconds
What the average says	199 ms, which sounds fine
What the p99 says	100 ms, because it cannot reach a 1 percent tail
What sees it	p99.9, or the maximum

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.

03 Three layers, and teams usually build one

INFRASTRUCTURE: brokers, disks, connections. Is Kafka alive.

PIPELINE: lag, rebalances, retries. Is work flowing.

BUSINESS: are payments completing, and are the results correct.

The outage where everything is green and payments are twenty minutes late lives entirely in the layers nobody built.

Broker and process level health signals report on the availability of the infrastructure, not on the progress or correctness of application work.

First principles, not vendor documentation

Kafka exposes broker and client metrics for monitoring, and the application layer is outside what those metrics can observe.

Kafka 4.3, Monitoring

04 Lag is one number telling two different stories

Consumer lag measures how far a consumer group progress is behind the latest available record on a partition. That is genuinely useful and it is not sufficient.

Lag climbing because consumers are slow and lag climbing because producers surged look identical on the graph. One needs investigation and one may be completely fine.

Worse, lag says nothing about whether the outcome was correct. A consumer that processes everything immediately and wrongly has zero lag.

Consumer lag measures the distance between a consumer group committed position and the latest available record on a partition.

Kafka 4.3, Monitoring

Identical lag can indicate slow consumers or a producer surge, and the metric alone cannot distinguish them.

First principles, not vendor documentation

05 Count and age answer different questions

A backlog of ten thousand records that clears in two seconds is harmless. A backlog of ten records that has sat there for two hours is an incident.

COUNT tells you how many. AGE tells you how long.

For payments, age is the one a customer actually feels, and it is the one most dashboards omit.

Alerting on count alone means the small permanent backlog never fires.

Lag count and lag age answer different questions, and a threshold on count cannot detect a small backlog that never clears.

First principles, not vendor documentation

The signal a customer experiences is elapsed time to completion, not the number of records outstanding.

Operational practice, not vendor documentation

06 The signal nobody builds

Lag is zero. Every event processed immediately. Every broker metric normal.

And once a day, one transfer creates an unbalanced financial journal.

That is not a health problem. It is a CORRECTNESS problem, and no infrastructure metric will ever see it. The check that catches it is one your team writes: do the debits equal the credits, and is that still true an hour later.

A correctness invariant violated during otherwise healthy processing produces no signal in infrastructure or pipeline metrics.

First principles, not vendor documentation

A continuous invariant check over business state is the only layer that can detect a wrong result produced promptly.

Operational practice, not vendor documentation



07 Retries are their own traffic class

Counting retries together with normal traffic hides both, because one is a small fraction of the other.

Watch them separately: how many are retrying now, which failure type dominates, how old the oldest retry is, how fast the dead letter topic is growing.

A retry system with no signals fails silently, which is the worst way for it to fail.

Retry and dead letter traffic measured together with normal traffic obscures both.

`Operational practice, not vendor documentation`

Dead letter topic growth rate is the signal that distinguishes an isolated bad record from a systematic failure.

`First principles, not vendor documentation`

08 Rebalances belong on the same timeline as lag

A lag spike at 14:02 that recovers by 14:05 looks like a traffic anomaly. It was a deploy, and a rebalance, and then catch up.

Without rebalance rate on the same dashboard, every deploy looks like a mystery, and a rebalance storm looks like variable traffic.

The consumer exposes rebalance rate and rebalance latency through its own metrics interface, alongside records-lag.

The consumer exposes rebalance rate and rebalance latency metrics through its own metrics interface, alongside records-lag.

`Kafka 4.3, Consumer Metrics`

Alerting on lag alone hides rebalance storms, because the lag signature of a rebalance resembles a traffic spike at typical dashboard resolution.

`Operational practice, not vendor documentation`

09 The one that is invisible while everything is green

The application is healthy. The database is healthy. Kafka is healthy. The outbox publisher has been stopped for six hours.

Nothing is lost. Nothing is delivered. Every dashboard is green.

ALERT ON THE AGE OF THE OLDEST UNPUBLISHED ROW. Nothing else in the system will tell you, because from every component point of view nothing is wrong.

A stopped relay between a database and Kafka produces no error in either system, because neither is failing.

First principles, not vendor documentation

The age of the oldest unpublished outbox row is the only local signal that detects a stopped relay.

First principles, not vendor documentation

10 Percentiles, and the aggregation error almost everyone makes

Ninety nine payments finish in 100 ms. One takes ten seconds. The average is 199 ms and says everything is fine.

THE CORRECTION MOST VERSIONS OF THIS NEED. The usual next line is "and the p99 shows it". With one bad request in a hundred, it does not: the p99 sits at the ninety ninth of a hundred sorted observations and the slow one is the hundredth. You need a percentile that can reach the tail you actually care about, which for a one in a hundred event means p99.9 or the maximum.

Choose the percentile for the frequency of the event, not for how senior the number sounds.

And the subtler one, worth saying out loud in an interview: if three instances each compute their own p95, AVERAGING those three does not give you the system wide p95. The maths does not work that way, and the error is not small.

Aggregate the underlying observations, not the summaries.

An arithmetic mean is insensitive to a small tail, while a percentile is defined by a position in the sorted observations.

Standard statistics, stated as such rather than as vendor behaviour

A percentile can only reach a tail more frequent than its own complement, so a one in a hundred event is invisible at p99 and visible at p99.9.

Standard statistics, stated as such rather than as vendor behaviour

Percentiles are not linear, so the mean of per-instance percentiles is not the percentile of the combined population.

Standard statistics, stated as such rather than as vendor behaviour

11 Never put a payment id in a metric label

One time series per payment, kept forever. High cardinality labels do not degrade gracefully: they take the metrics backend down.

A label is for a BOUNDED set of values. Result success, retry, dead letter. Failure class. Topic name.

When you need to follow one payment, that is what a trace or a log is for. It was never a metrics question.

Every unique combination of label values creates a new time series, so labels should not be used for unbounded sets of values such as identifiers.

Prometheus, Metric and Label Naming

Following one specific request is a tracing or logging concern, not a metrics concern, because metrics are aggregates by construction.

First principles, not vendor documentation

12 The test an alert must pass

If nobody would do anything about it, it is not an alert.

Would it still fire during an ordinary deploy? If it fires at three in the morning, is that justified?

Alerts nobody acts on train people to ignore the ones that matter, and that training is very hard to undo.

An alert with no associated action produces alert fatigue, which reduces response to the alerts that do carry one.

Operational practice, not vendor documentation

An alert that fires during routine operations such as a deploy is measuring the operation rather than a fault.

First principles, not vendor documentation

13 The first dashboard, in order

Lag AGE per consumer group. Rebalance rate. Retry depth and dead letter growth. Age of the oldest unpublished outbox row. Payments completed, with a p99.

Five panels, not fifty. A dashboard nobody reads is worse than no dashboard, because it looks like coverage.

And one invariant check running continuously over the business state, because that is the only thing on this list that can catch a wrong answer delivered on time.

A small number of panels chosen for what they catch is more effective than a large number chosen for what is available to plot.

`Operational practice, not vendor documentation`

Questions to ask before you answer

1. Which of the three layers does this metric belong to?
2. Is lag being measured as a count, an age, or both?
3. What would tell you a correct-looking result was wrong?
4. Is retry traffic separated from normal traffic?
5. Is rebalance rate on the same timeline as lag?
6. Are percentiles computed from observations or averaged from summaries?
7. Does any label hold an unbounded set of values?
8. For each alert: what would somebody actually do?

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

Facebook

<https://www.facebook.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>

Buy me sadza

<https://buymeacoffee.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Apache Kafka and the Kafka logo are trademarks of the Apache Software Foundation. PostgreSQL and its logo are trademarks of the PostgreSQL Community Association. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.